

# CS-405: Deep Learning

From Word Embeddings to Attention: RNNs for Classification,  
Tagging, and Neural Machine Translation

Dr. Sajjad Hussain

SEEDS, NUST

April 14th, 2026

# Lecture Outline

1 RNNs for Language Tasks

2 Attention Mechanism

# From Word Embeddings to Sequential Models

## Recap: Word Embeddings

We learned to represent words as dense vectors (embeddings) that capture semantic meaning.

**But word embeddings alone are not enough for language tasks!**

### The Problem:

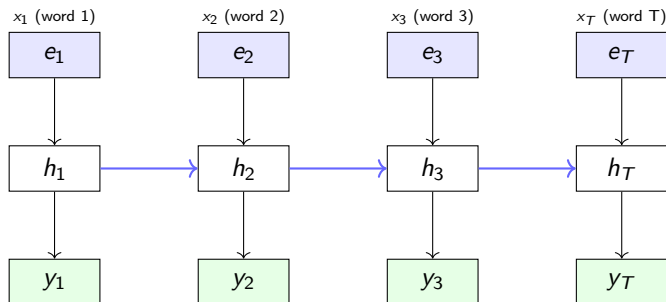
- Sentences have **order**
- Meaning depends on **sequence**
- "Dog bites man"  $\neq$  "Man bites dog"

### The Solution:

- Process words **in order**
- Maintain **memory** of previous words
- Recurrent Neural Networks (RNNs)

**Today:** How RNNs use word embeddings for language tasks

# RNN Architecture for Language Tasks



## How it works

- Each word  $x_t \rightarrow$  embedding  $e_t$  (from embedding matrix)
- Hidden state  $h_t = f(W \cdot h_{t-1} + U \cdot e_t + b)$
- $h_t$  contains **memory of all previous words**
- Output  $y_t$  depends on current hidden state

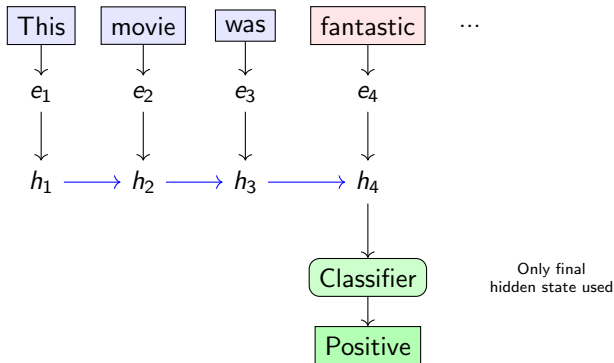
# Task 1: Sentiment Classification

## Goal

Classify entire sentence as Positive, Negative, or Neutral.

## Example:

"This movie was absolutely fantastic!" → **Positive**



# Sentiment Classification: Detailed Example

## Input Sentence

"I love this product, it's amazing!"

## Step-by-step processing:

| Step | Word     | Hidden State Contains     |
|------|----------|---------------------------|
| 1    | I        | Information about "I"     |
| 2    | love     | "love" + previous context |
| 3    | this     | "I love this"             |
| 4    | product  | "I love this product"     |
| 5    | it's     | Full sentence so far      |
| 6    | amazing! | Complete context!         |

## Final hidden state $h_6 \rightarrow$ Classifier:

$$\text{Score} = W_{\text{class}} \cdot h_6 + b_{\text{class}}$$

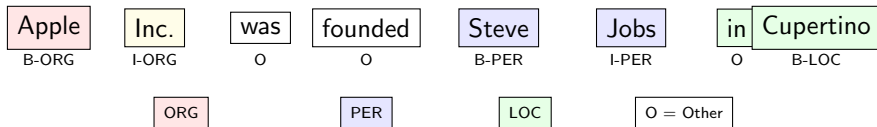
$$P(\text{Positive}) = 0.92, P(\text{Negative}) = 0.05, P(\text{Neutral}) = 0.03$$

## Task 2: Named Entity Recognition (NER)

### Goal

Label **each word** with entity type (Person, Organization, Location, etc.)

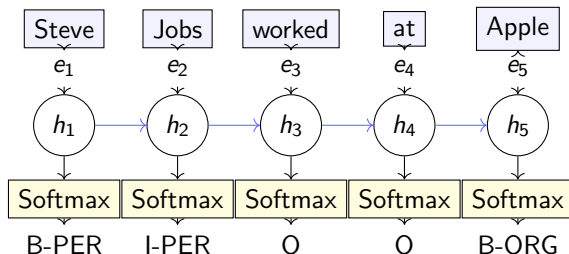
### Example:



### Key Difference from Sentiment Analysis

- Need **prediction at EVERY position**
- Each hidden state  $h_t \rightarrow$  classifier for token  $t$
- Context from **both sides** helps (bidirectional RNN is better!)

# NER: Per-Token Predictions with RNN



## Important

Each output is computed from its **local hidden state**, which contains both left and (some) right context (in bidirectional RNNs).



# Machine Translation: A Many-to-Many Task

## The Challenge

Translate a sentence from source language (French) to target language (English):

“Je suis étudiant” → “I am a student”

**Input and output lengths can be DIFFERENT!**

### Sequence Classification:

- Input: sequence
- Output: single label
- Many-to-one

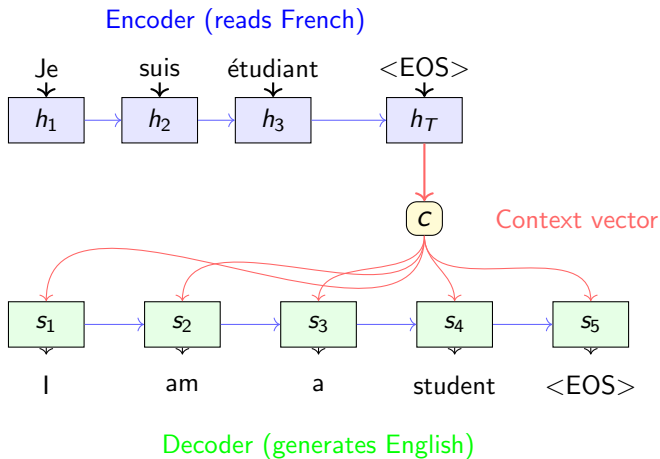
### NER / POS Tagging:

- Input: sequence
- Output: same length
- Many-to-many (synchronous)

### Machine Translation:

- Input: sequence (French)
- Output: sequence (English)
- Output length may differ!
- Translation only starts AFTER entire input is read

# Encoder-Decoder Architecture



## Key Insight

Encoder compresses entire input into a **context vector**  $c$ . Decoder generates output from this vector.

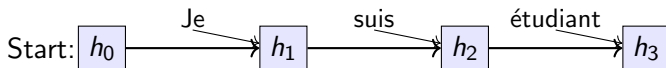
# How the Encoder Works

Encoder: Reads Input One Token at a Time

$$h_t = \text{RNN}(h_{t-1}, e(x_t))$$

where  $e(x_t)$  is the word embedding for input token  $x_t$

**Example: Translating "Je suis étudiant" (French → English)**



**Step-by-step:**

- 1  $h_1 = \text{RNN}(h_0, e(\text{Je})) \rightarrow$  encodes "Je"
- 2  $h_2 = \text{RNN}(h_1, e(\text{suis})) \rightarrow$  encodes "Je suis"
- 3  $h_3 = \text{RNN}(h_2, e(\text{étudiant})) \rightarrow$  encodes entire sentence

Final hidden state  $h_3$  becomes the context vector  $c$ !

# How the Decoder Works

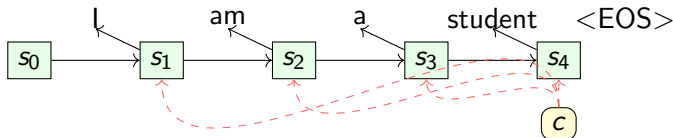
Decoder: Generates Output One Token at a Time

$$s_t = \text{RNN}(s_{t-1}, [e(y_{t-1}); c])$$

$$P(y_t | y_{<t}, c) = \text{softmax}(W \cdot s_t + b)$$

# How the Decoder Works

## Generation Process for "I am a student":



- 1  $s_1$  takes  $c$  and start token  $\rightarrow$  predicts "I"
- 2  $s_2$  takes  $s_1$  and "I"  $\rightarrow$  predicts "am"
- 3  $s_3$  takes  $s_2$  and "am"  $\rightarrow$  predicts "a"
- 4  $s_4$  takes  $s_3$  and "a"  $\rightarrow$  predicts "student"
- 5  $s_5$  takes  $s_4$  and "student"  $\rightarrow$  predicts  $\langle \text{EOS} \rangle$

# Word Embeddings in Translation

## Dual Embedding Spaces

Translation requires TWO embedding matrices:

### Source Embeddings (French):

- Map French words to vectors
- Used by Encoder
- $E_{src} \in \mathbb{R}^{|V_{src}| \times d}$

### Target Embeddings (English):

- Map English words to vectors
- Used by Decoder
- $E_{tgt} \in \mathbb{R}^{|V_{tgt}| \times d}$

### Example:

| French Word | Embedding                  | English Word |
|-------------|----------------------------|--------------|
| Je          | $[0.2, -0.5, 0.3, \dots]$  | I            |
| suis        | $[-0.1, 0.4, -0.2, \dots]$ | am           |
| étudiant    | $[0.3, -0.1, 0.6, \dots]$  | student      |

These embeddings are **LEARNED** during training!

# Training the NMT Model

## Training Data

Parallel corpus: (French sentence, English translation) pairs

### Example:

(“Je suis étudiant”, “I am a student”)

### Teacher Forcing during training:

- 1 Feed French sentence to Encoder  $\rightarrow$  context vector  $c$
- 2 Feed English sentence to Decoder (shifted by 1, with  $\langle \text{SOS} \rangle$  start)
- 3 Compare Decoder output with actual next word
- 4 Compute cross-entropy loss at each position
- 5 Backpropagate through both Encoder and Decoder

# Inference: Translating New Sentences

No teacher forcing during inference!

The model must generate using its own predictions.

## Greedy Decoding (simplest):

- 1 Encoder reads French sentence  $\rightarrow$  context vector  $c$
- 2 Decoder starts with  $\langle \text{SOS} \rangle$  and  $c \rightarrow$  predicts first English word
- 3 Feed predicted word as next input
- 4 Repeat until  $\langle \text{EOS} \rangle$  is generated

## Example: Translating "Je mange une pomme"

| Step | Decoder Input                               | Predicted Output             |
|------|---------------------------------------------|------------------------------|
| 1    | $\langle \text{SOS} \rangle$                | I                            |
| 2    | $\langle \text{SOS} \rangle$ I              | eat                          |
| 3    | $\langle \text{SOS} \rangle$ I eat          | an                           |
| 4    | $\langle \text{SOS} \rangle$ I eat an       | apple                        |
| 5    | $\langle \text{SOS} \rangle$ I eat an apple | $\langle \text{EOS} \rangle$ |



# The Problem with Fixed Context Vectors

## Recall: Basic Encoder-Decoder

The encoder compresses the ENTIRE source sentence into a **single context vector**  $c$ .

### Limitations:

- All source information compressed into ONE vector
- Long sentences lose early information
- Decoder cannot "look back" at specific source words

**Solution: Attention Mechanism allows looking back!**

# Intuition: Human Translation with Attention

## How Humans Translate Long Sentences

When translating, we don't memorize the entire sentence at once. We **pay attention** to relevant parts as we go.

### Example: Translating French to English

*"Le chat noir qui était assis sur le tapis a soudainement couru"*

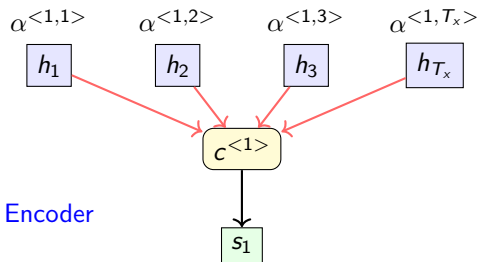
| Generating English Word | Paying Attention To |
|-------------------------|---------------------|
| "The"                   | "Le"                |
| "black"                 | "noir"              |
| "cat"                   | "chat"              |
| "was sitting"           | "était assis"       |
| "on the rug"            | "sur le tapis"      |
| "suddenly"              | "soudainement"      |
| "ran"                   | "couru"             |

# Attention Mechanism: The Big Picture

## Key Idea

At each decoder step, compute a **weighted sum** of ALL encoder hidden states.

$$c^{<t>} = \sum_{j=1}^{T_x} \alpha^{<t,j>} \cdot h_j \quad \sum_{j=1}^{T_x} \alpha^{<t,j>} = 1, \quad \alpha^{<t,j>} \geq 0$$



Encoder

Decoder

# Computing Attention Weights $\alpha^{<t,j>}$

## Step 1: Compute Alignment Scores

For decoder step  $t$  and each encoder hidden state  $h_j$ :

$$e^{<t,j>} = \text{score}(s_{t-1}, h_j)$$

Common scoring function is **Bilinear**:  $s_{t-1}^\top W_a h_j$

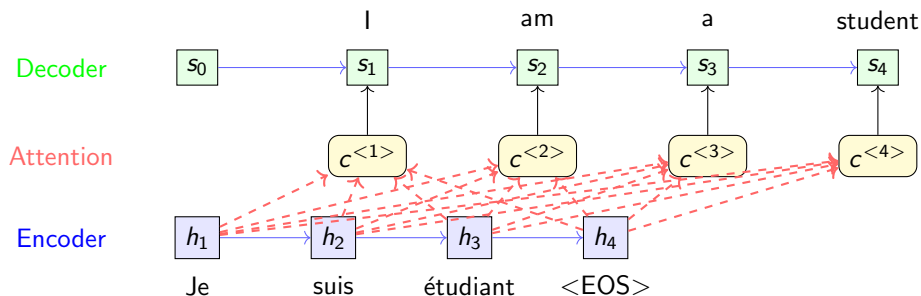
## Step 2: Normalize with Softmax

$$\alpha^{<t,j>} = \frac{\exp(e^{<t,j>})}{\sum_{k=1}^{T_x} \exp(e^{<t,k>})}$$

## Intuition:

- $e^{<t,j>} =$  How well does encoder state  $h_j$  "match" decoder state  $s_{t-1}$ ?
- $\alpha^{<t,j>} =$  Softmax converts scores to probabilities (sum = 1)

# Complete NMT with Attention Architecture



## Forward Pass Equations

$$\text{Encoder: } h^{<t>} = \text{RNN}_{enc}(h^{<t-1>}, x^{<t>}) \quad (1)$$

$$\text{Attention: } e^{<t',t>} = \text{score}(s^{<t'-1>}, h^{<t>}) \quad (2)$$

$$\alpha^{<t',t>} = \frac{\exp(e^{<t',t>})}{\sum_{k=1}^{T_x} \exp(e^{<t',k>})} \quad (3)$$

$$c^{<t'>} = \sum_{t=1}^{T_x} \alpha^{<t',t>} h^{<t>} \quad (4)$$

$$\text{Decoder: } s^{<t'>} = \text{RNN}_{dec}(s^{<t'-1>}, [y^{<t'-1>}; c^{<t'>}]) \quad (5)$$

$$\hat{y}^{<t'>} = \text{softmax}(W_y s^{<t'>} + b_y) \quad (6)$$

# Mathematical Formulation

| Symbol            | Meaning                                           |
|-------------------|---------------------------------------------------|
| $h^{<t>}$         | Encoder hidden state at time $t$                  |
| $s^{<t'>}$        | Decoder hidden state at time $t'$                 |
| $\alpha^{<t',t>}$ | Attention weight from encoder $t$ to decoder $t'$ |
| $c^{<t'>}$        | Context vector for decoder step $t'$              |
| $y^{<t'-1>}$      | Previously generated output word embedding        |

# Attention Matrix: Visualizing All Weights

## Attention Weight Matrix

$$A = \begin{bmatrix} \alpha^{<1,1>} & \alpha^{<1,2>} & \dots & \alpha^{<1,T_x>} \\ \alpha^{<2,1>} & \alpha^{<2,2>} & \dots & \alpha^{<2,T_x>} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha^{<T_y,1>} & \alpha^{<T_y,2>} & \dots & \alpha^{<T_y,T_x>} \end{bmatrix}$$

Each row sums to 1:  $\sum_j \alpha^{<t,j>} = 1$

**Example: Translating "Je suis étudiant" → "I am a student"**

| Decoder (English) | Encoder (French) |             |             | Row Sum |
|-------------------|------------------|-------------|-------------|---------|
|                   | Je               | suis        | étudiant    |         |
| I                 | <b>0.70</b>      | 0.20        | 0.10        | 1.00    |
| am                | 0.10             | <b>0.80</b> | 0.10        | 1.00    |
| a                 | 0.05             | 0.10        | <b>0.85</b> | 1.00    |
| student           | 0.02             | 0.03        | <b>0.95</b> | 1.00    |



# Why Attention Works: Summary

## Without Attention:

- Single context vector  $c$
- Same  $c$  for all decoder steps
- Early words easily forgotten
- Poor performance on long sentences

## Key Benefits:

- 1 Solves the bottleneck problem
- 2 Provides interpretability (see what model focuses on)
- 3 Gradients flow directly to relevant encoder states
- 4 Essential building block for Transformers!

## With Attention:

- Dynamic  $c^{<t>}$  for each decoder step
- Different attention for different outputs
- Can "look back" at any source word
- Much better on long sentences